

Policy

The policy module is module ID 5. It can evaluate policies against supported numeric GPU fields and retain the resulting violations. The older group policy interface remains available for compatibility. The module is loaded when policy functionality is first requested.

When policy is useful

Policy is useful when an operator knows which field and condition matter and wants DCGM to retain violations for polling or callback delivery. For response logic—such as draining a scheduler node, resetting a GPU, running diagnostics, paging an on-call engineer, or correlating several metrics—an external controller should consume DCGM health, fields, or policy notifications and perform that automation.

How policy fits with other features

Policy, health monitoring, and diagnostics are related but not interchangeable:

- [Health Monitoring](#) evaluates a broad set of retained telemetry and returns a current health summary when queried.
- Field-based policy compares supported numeric fields with configured operators and records matching violations. The legacy policy interface watches seven fixed conditions.
- [Diagnostics](#) actively exercises an available GPU when invoked explicitly by an operator or management integration.

Field-based policies

A field-based policy identifies a numeric field, an operator, and, except for the changed-value operator, a threshold. It can target all GPUs or a list of GPU entity IDs. Operators are `>`, `>=`, `<`, `<=`, `==`, `!=`, and `changed`. Policies can use console, file, or callback notification channels, and a per-entity rate limit controls repeated notifications.

For example, create a temperature policy for every GPU and watch for its violations:

```
$ dcgmi policy --create --field 150 --name "GPU temperature warning" \  
  --operator '>' --threshold 85 --entities all  
$ dcgmi policy --watch --field 150
```

Use `--list` to discover the generated policy ID. The field ID and policy ID together select a policy for get, modify, delete, enable, and disable operations. `--import` replaces the host engine's complete

field-policy registry from YAML rather than merging entries. DCGM 4.7 can replace the registry with an empty or partially valid import, so validate the file and review the resulting list. Field policies are persisted by the host engine and restored when the policy module loads.

See [dcgmi policy](#) for the complete operation, parameter, YAML, polling, and JSON-output reference, and [Policies](#) for the programmatic interface.

Moving from legacy group policies

Existing automation can continue to use the legacy group policy interface, which watches seven fixed condition categories. For new automation, prefer a field-based policy because it identifies the exact numeric field and comparison being evaluated and can retain violations without an attached `dcgmi` listener.

When migrating, map each legacy condition to the field and threshold your site actually intends to enforce, create the field policy, and compare its notifications with the old workflow before removing the legacy policy. Keep recovery in a scheduler-aware consumer: legacy action and validation values are stored for compatibility but do not reset a GPU or run a diagnostic.

See [DEPRECATED LEGACY POLICY](#) for every legacy option, condition, state rule, and notification behavior.

Further reading

- [dcgmi policy](#) is the exhaustive reference for field-policy operations and legacy policy behavior.
- [dcgmi group](#) documents group creation and membership.
- [dcgmi health](#) and [dcgmi diag](#) document the adjacent health and diagnostic workflows.
- [Policies](#) documents programmatic policy configuration and callbacks.

[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.